Ticket To Ride
Design Document

Authors:
Ng Zi Wei (33590877)
Hew Jia Jing (33589739)
Yu Jia Ming (33592586)

Team: Solution Seekers

Document Revision History

Date	Version	Description	Author
10/25/2026	0.0	Added initial Design Document	Yu Jia Ming
09/05/2026	0.1	Added initial Introduction and Team	Hew Jia Jing
28/05/2026	0.2	Added initial UML class diagram and initial design rationales	Hew Jia Jing
03/05/2026	0.3	Improve design rationale	Ng Zi Wei, Yu Jia Ming, Hew Jia Jing
04/05/2026	0.4	Improve design rationale	Ng Zi Wei, Yu Jia Ming, Hew Jia Jing

Table of Contents

1 Introduction 4
2 Team Information 5
2.1 Team Name and Membership 5
2.2 Team Schedule 5
2.3 Team Contribution 6
2.4 Link to the Gitlab repository 6
3 Object-Oriented Design 7
4 Design Rationales 10
4.1 Three Key Classes 10
4.2 Three Key Relationships 13
4.3 Inheritance Decisions 13
4.4 Cardinality Decisions 14
4.5 Design Patterns Used 14
4.5.1 Command Pattern 14
4.5.2 Factory Pattern 15
4.5.3 Visitor Pattern 16
4.5.4 Strategy Pattern 17
4.5.5 Mediator Pattern 18
4.5.6 Builder Pattern 19
4.6 Architectural Trade-offs: MVC Boundaries & Data Transfer 20
5 Supplementary Documentation 20
5.1 Test Performed 20
5.1.1 Integration Test 20
5.1.2 Unit Test 22
5.2 Known Defects 22
5.3 Implementation Limitation 22
5.5 Executable Deliverable 22
6 AI declaration 22

1 Introduction

This document provides a comprehensive overview of the technical design software and architecture for the Ticket To Ride: London Edition game, allowing the development team to develop the system based on this document. This document outlines the overall system design, including the structure and rationale behind the design decisions.

The main problem addressed in this project is to translate a complex physical board game into a digital format, while maintaining its official rules. The system must handle core gameplay elements including train and destination card distribution, player's available actions, validation of player actions and end-game scoring, delivered through an interactive graphical interface.

This document is structured into the following sections:

- **Section 2: Team Information:** An overview of the development team, including member roles, professional strengths, sprint meeting schedules, workload distribution, and visual evidence of repository contributions.

- **Section 3: Object-Oriented Design:** The structural design of the system presented through UML class diagrams. This section defines key classes, attributes, methods, relationships, and cardinalities, alongside some draft iterations of the design.

- **Section 4: Design Rationales:** Justification of design decisions made in Section 3, including the reasoning of the inclusion of specific classes, relationships and cardinalities and design patterns applied.

- **Section 5: Supplementary Documentation:** Additional resources and references.

2 Team Information

2.1 Team Name and Membership

Personal Team Name: Solution Seekers

Moodle Team Name Mapping: This team corresponds to A2_MA_Friday02PM_08 in the Moodle Sprint 2 submission page.

Name	Email	Role	Technical Strengths	Professional Strengths
Ng Zi Wei	zngg0041@student.monash.edu	Team Lead, Developer	Object-oriented design, debugging, code quality awareness	Team coordination, detail-oriented, team collaboration
Hew Jia Jing	jhew0010@student.monash.edu	Developer	Problem-solving, bug tracking, collaborative coding, UI design	Team collaboration, adaptability
Yu Jia Ming	jyuu0107@student.monash.edu	Developer	Object-oriented design, problem solving, debugging	Analytical thinking, teamwork

2.2 Team Schedule

Meeting Schedule:

Our team followed the schedule below to hold meetings:

Meeting Type	Day & Time	Host	Purpose
Sprint Workshop	At the start and end of sprint	In-person	Deliverable review and sprint planning
Weekly Stand up	Three times per week (Mon/Wed/Fri)	Zoom/WhatsApp	Progress updates and blockers
Ad hoc session	If required	Zoom/WhatsApp	Bug fixes and urgent discussions

All members attended the scheduled sessions. Communication between meetings is primarily conducted through the team's WhatsApp group to ensure timely updates.

Weekly stand-up meetings are typically held on Monday (12:00 pm), Wednesday (12:00 pm), and Friday (2:00 pm), with each session lasting approximately 15-20 minutes.

Regular Work Schedule:

Each team member dedicated approximately 7 hours per week (averaging 1 hour per day) on individual work. However, this schedule is flexible to accommodate individual timetables while maintaining consistent progress on the project.

During these individual sessions, team members focused on their assigned technical tasks, including writing and refining code, testing, debugging and updating the documentation. This time is also used to prepare feedback for upcoming group meetings. This approach ensures that individual tasks are properly progressed, allowing all members to make meaningful contributions to the project.

Workload distribution:

In Sprint 2, the team moved away from a strict ticket-based tracking system and adopted a more flexible, communication-driven workflow to better fit the fast-paced implementation phase. Main objectives were discussed during in-person Sprint Workshops, while tasks were dynamically assigned based on each member's technical strengths and availability. Instead of relying heavily on story points, workload balance was managed through continuous communication, where members focused on one main task at a time before taking on new responsibilities after their current work was merged.

Task tracking and coordination were mainly handled through our active WhatsApp group chat. Progress updates, implementation status, bug fixes, and documentation work were shared in real time, allowing the team to quickly resolve blockers, discuss design decisions, and redistribute tasks whenever needed during our tri-weekly stand-ups and daily discussions.

2.3 Team Contribution

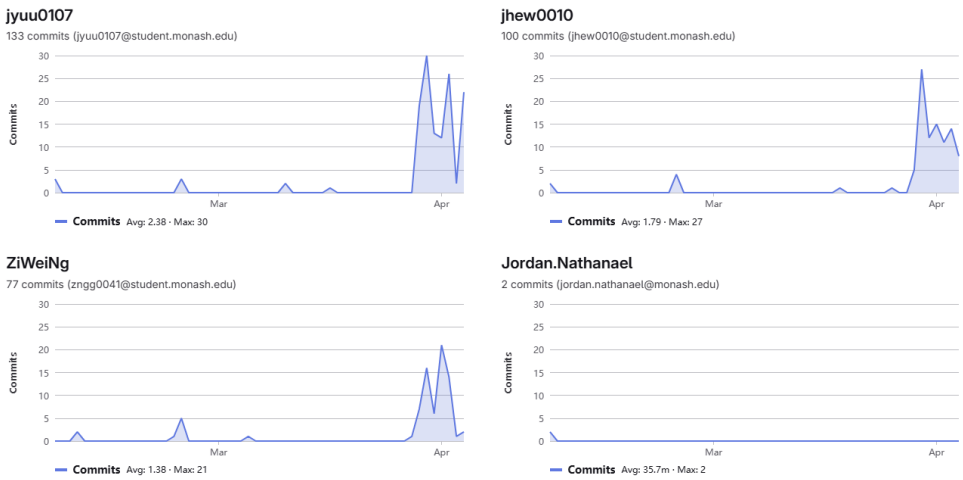


Figure 1: Contributor Analytics

2.4 Link to the Gitlab repository

https://git.infotech.monash.edu/FIT3077/fit3077-s1-2026/assignment-groups/MA_Friday02PM_08

3 Object-Oriented Design

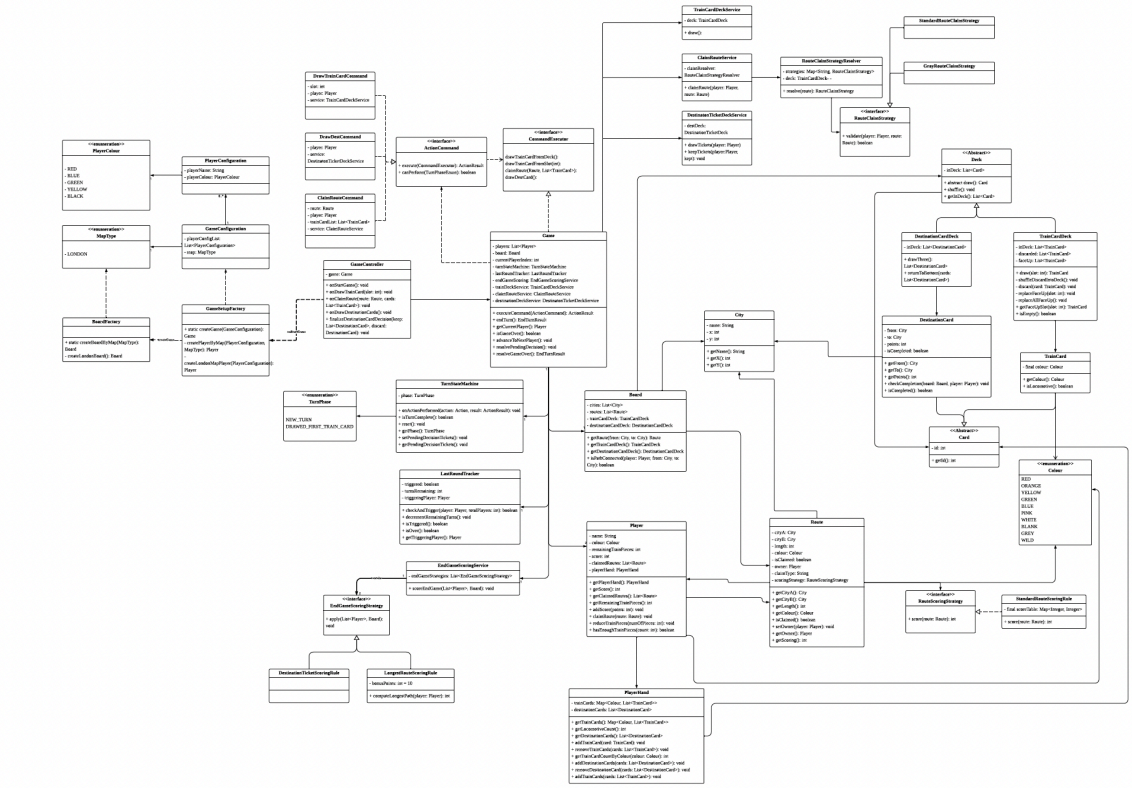


Figure 2: Iteration 4 of Class Diagram of the System

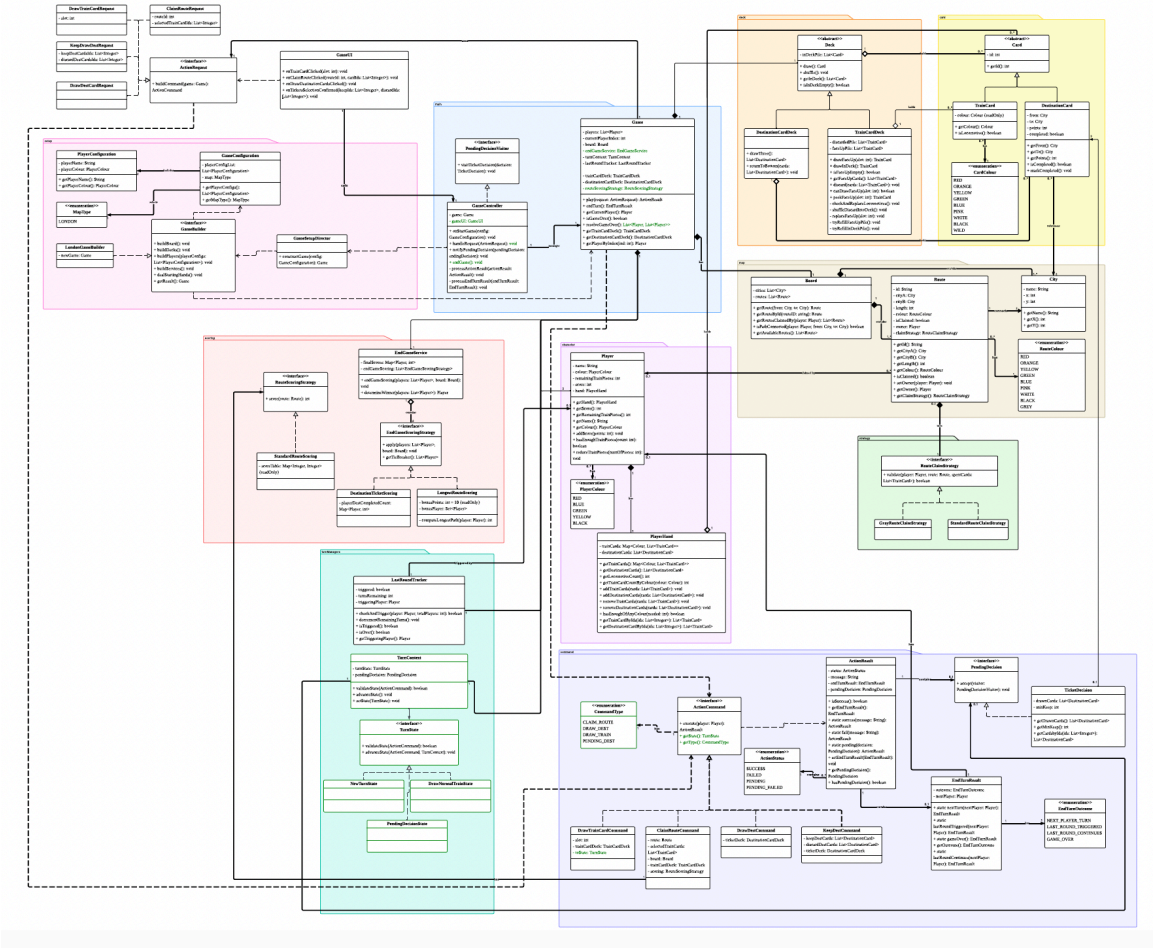


Figure 3: Iteration 6 of Class Diagram of the System

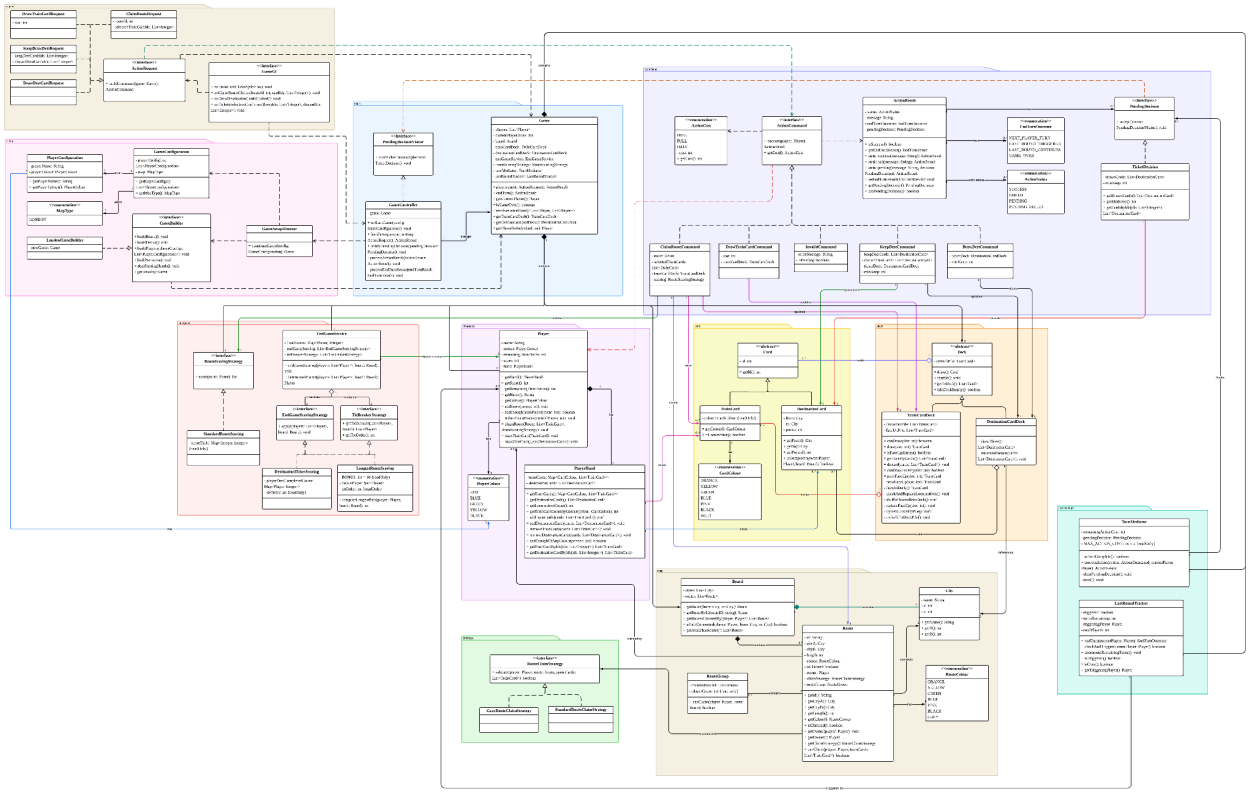


Figure 4: Final Class Diagram of the System

4 Design Rationales

4.1 Three Key Classes

Class 1: PlayerHand

Alternative	**A:** Store cards directly in Player	**B:** Create a PlayerHand class
Advantage	- Simpler design with fewer classes - Direct access to card data	- Encapsulates all inventory/card logic in one place - Improves extensibility, easier to add new resources without touching Player - Easier to test card logic independently - Keeps Player focused purely on identity and score (SRP)
Disadvantage	- Violates Single Responsibility Principle (SRP) - May become God class as Player becomes bloated in managing identity and inventory/card management - Harder to maintain and extend when card logic grows	- Introduces one additional class - Requires delegation between Player and PlayerHand

Decision: Alternative B is chosen. This is because managing a player's inventory involves multiple operations, such as counting, adding and validating different cards, and this is likely to grow in complexity. Integrating these methods directly into Player would overload it and reduce maintainability. Implementing a separate class keeps responsibilities clearly defined between Player and PlayerHand, adhering to the Single Responsibility Principle (SRP).

Class 2: LastRoundTracker

Alternative	**A:** Store end-game variables directly in Game	**B:** Create a LastRoundTracker class
Advantage	- Fewer classes to manage - Simpler implementation without delegation required	- Encapsulates last-round triggered state in one place - Keeps Game focused purely on overall game flow (SRP) - Makes threshold logic explicit and easier to test independently

		- Easier to extend if last round rules grows
Disadvantage	- Violates Single Responsibility Principle (SRP) by giving Game more responsibilities - Last-round triggering rules become scattered across multiple Game methods - Harder to test last game round behaviour independently	- Introduces one additional class - Requires the Game to collaborate/delegate to LastRoundTracker at the end of every turn

Decision: Alternative B is chosen. The last-round/end-game mechanic has its own distinct state (e.g., triggered, turnsRemaining, triggeringPlayer) and behaviour. Because these variables must persist across multiple players' turns, they cannot simply be temporary method variables. It is cleaner to encapsulate this cohesive state inside a LastRoundTracker instead of adding extra flags and counters into Game. This also better adheres to the Single Responsibility Principle (SRP), keeping Game focused on overall game flow rather than handling detailed countdown logic.

Class 3: EndGameService

Alternative	**A:** Put final scoring and winner determining directly in Game	**B:** Create an EndGameService class
Advantage	- Simpler initial design with fewer classes - Direct access to Player and Board data without needing delegation	- Separates the "final scoring" phase from normal gameplay loops (SRP) - Easily extensible; different end-game scoring or tie-breaker strategies can be plugged in without modifying core game logic - Easier to test complex winner calculation math independently
Disadvantage	- God class risk as Game becomes bloated by managing game flow and complex end-game math. - Violates OCP as adding new scoring rules requires modifying the Game class - Harder to test winner calculation independently	- Introduces an additional service class. - Requires the Game to delegate data to the service at the end of the game

Decision: Alternative B is chosen. Because final scoring is a distinct phase of the game, embedding it within the Game class violates the Single Responsibility Principle (SRP). Delegating this responsibility to an EndGameService achieves high cohesion, making the final scoring logic and winner selection much easier to maintain, test and extend. Any new final scoring and tie breaker only require adding a new class implementing the respective strategy interface without any modification needed to EndGameService and Game class, highly promoting Open-Closed Principle (OCP).

All three decisions were made to separate distinct responsibilities into dedicated components, so that it can be easier to extend, test and maintain as complexity grows.

4.2 Three Key Relationships

Relationship 1: The relationship between PlayerHand and Player is effectively composition because a hand only exists as part of a Player during the game. The lifecycle of PlayerHand depends on the Player. This cannot be aggregation because the hand can never exist independently.

Relationship 2: The relationship between Board with Route and City is modelled as composition because routes and cities define the board. They will have no meaning outside of it. If the board is destroyed, they should also be destroyed. Therefore, this suggests that the board is a strong owner of them.

Relationship 3: The relationship between Route and Player is simple association because a route may or may not be owned by a player, and players exist independently of routes. Routes do not control the lifecycle of the player. Therefore, this suggests a loose relationship.

4.3 Inheritance Decisions

Cards & Decks

Alternative	**A:** Create abstract base classes (Card and generic Deck<T extends Card>	**B:** No inheritance (Independent classes for all cards and decks)
Advantage	- Avoids duplication of shared attributes and methods logic. - Supports future expansion decks and cards without code duplication. - Using <T extends Card> prevents adding invalid objects to a deck.	- Simpler overall design - Cards and Decks are not tightly coupled to shared base implementation
Disadvantage	- Adds an extra layer of abstraction - Concrete cards and decks must adhere to the base Deck's internal data structure - Slight over-engineering, as the	- Code duplication, shared attributes and complex deck mechanics (shuffling/drawing) must be rewritten for every new type - Less clear conceptual model and a more rigid architecture

	system rarely uses true polymorphic collections (e.g., List<Deck>)	

Decision: Alternative A is chosen. By combining the inheritance strategy for both Cards and Decks, a highly cohesive and reusable architecture is created. Creating an abstract Deck<T extends Card> allows centralizing complex management logic and eliminating severe code duplication across concrete decks. This follows the DRY principle and also better supports the Open/Closed Principle (OCP), since new card or deck types can be added later without modifying the existing inventory system.

4.4 Cardinality Decisions

Cardinality 1: A Route is modelled to be claimed by 0..1 Player while a Player has 0..* Route because a route starts as unclaimed, so it may have no owner, but once claimed, it will belong to exactly one player. And a player starts with no routes being claimed by him, and can claim as many routes as he is capable of claiming.

Cardinality 2: A game has 2 players because the game requires only two players to start for this sprint.

4.5 Design Patterns Used

4.5.1 Command Pattern

Application 1: Action Handling

Alternative	**A:** Centralized Routing (Double Dispatch via CommandExecutor and Services)	**B:** Command Pattern with Injected Receivers (ActionCommand)
Advantage	- Game acts as a clear, centralized controller for all action flow - Command classes are extremely thin, acting only as triggers	- Encapsulates each action as a discrete object - Precise scope, injects only the necessary receivers into commands, protecting other game states - Highly extensible as new commands can be added without modifying any existing interfaces or routing logic - It allows future features such as undo, replay to be added easily
Disadvantage	- The CommandExecutor interface hardcodes routing methods for every possible action - Adding a new action strictly requires modifying the core	- Slightly higher initial complexity - Results in “fatter” Action classes as they must store their receivers locally

	interface and updating Game to route that action to the correct service, violating OCP - Violates SRP since Game should be only responsible for game coordination and not action routing - Design is less scalable when number of actions increase - Commands are not self-contained, so they do not represent the Command Pattern strongly	

Decision: Alternative B is chosen. Alternative A's double dispatch design turned Game into a severe bottleneck; adding new actions required modifying core interfaces, violating the Open/Closed Principle (OCP). Alternative B fixes this using a true Command Pattern. By injecting specific receivers directly into self-contained commands, execution is reduced to a simple execute() call. This allows new actions to be added instantly without touching existing code. Although this approach introduces slightly more complexity and larger command classes, it improves extensibility, maintainability, testability, and separation of concerns.

4.5.2 Factory Pattern

Application 1: Action Creation

Alternative	**A:** Polymorphic Creation (ActionRequest)	**B:** Centralized Creation in Controller
Advantage	- Adheres to OCP, adding new actions only requires creating a new Request class; the Controller and Game is left completely untouched. - High cohesion, as the logic to translate raw data into an executable command is encapsulated inside the specific request itself - Eliminates ugly instanceof checks or large switch statements	- Simpler architecture with few classes
Disadvantage	- Introduce coupling to Game, the ActionRequest interface inherently depends on the Game class to fetch the necessary receivers during command construction - More classes will be required to maintain as each action needs its	- Every new action forces the modification of the Controller's logic (adding new methods or switch branches), violating OCP - Controller becomes responsible for both routing traffic and complex command object construction,

	own request class	violating SRP - The Controller must constantly pull internal data from Game to inject receivers, breaking encapsulation and introducing higher feature envy - Design is much harder to scale as the number of actions increases

Decision: Alternative A is chosen. Alternative B forces Controller to constantly pull internal data from Game to inject receivers, introducing feature envy. It also violates the Open/Closed Principle (OCP), as every new action would require modifying the Controller's switch statements or appending more methods. Alternative A encapsulates the logic and ensures the system is easier to extend without touching existing Controller code. Although this introduces coupling between ActionRequest and Game, this trade-off is acceptable because it produces higher cohesion, better extensibility and cleaner action creation logic.

4.5.3 Visitor Pattern

Application 1: Pending Decision for Player to choose DestinationCard to keep

Alternative	**A:** Encapsulate the logic inside DrawDestCommand	**B:** Separate the keeping logic into different command, Controller implement PendingDecisionVisitor to visit different PendingDecision
Advantage	- Easier to implement - Simple command flow - Faster to implement	- Strictly adhere to Single Responsibility Principle, drawing and keeping destination card splitted into distinct command - UI prompting logic handled by Controller via Visitor pattern to call appropriate UI method, allowing backend free from frontend logic - New decision types extend the system by adding a new PendingDecision and a new visitX() method in Visitor, without modifying existing commands
Disadvantage	- Exposure of frontend logic (UI) in backend for prompting user to select the cards to keep, introducing tighter coupling between game logic and frontend interaction flow - Violates Single Responsibility Principle as DrawDestCommand will be holding responsibility of drawing	- Adding new PendingDecision type requires adding a new method in the visitor interface, so this have an OCP trade-off

	and keeping cards - Do not have any natural extension point, where new decision workflow or multi-step action command require duplication of entire pattern	

Decision: Alternative B is chosen. Alternative A is faster and simpler to implement but it places too much responsibility inside DrawDestCommand, from managing draw destination card action to decision making and trigger UI prompting. This exposes frontend concerns inside backend logic, violating SRP and making the entire codebase to duplicate this pattern for new decision workflow commands. Alternative B makes a cleaner long term design, as it keeps backend logic separated from frontend logic, and creates a reusable structure for future decisions.

Although Alternative B has an OCP trade-off in Visitor interface as a new method needs to be added for each new decision-type workflow, introducing risk of visitor interface method explosion when more decision-type commands are introduced. This is acceptable as only the Controller is implementing the interface, where the interface methods are used to decide which UI method to be called which are the core responsibility of Controller, a coordination layer between frontend and backend, making OCP impact only affects Controller. Most importantly, this design introduced a reusable foundation for future new decision-type workflow commands without modifying existing classes except for Visitor interface, while maintaining extensibility, testability, and maintainability.

4.5.4 Strategy Pattern

Application 1: Route Claiming

Alternative	**A:** Centralized Resolver (ClaimRouteService + Resolver)	**B:** Strategy Injected into Route Object
Advantage	- Centralizes all validation logic in one service layer - Less validation logic in Route objects	- High cohesion, the behaviour for claiming a Route is bound directly to the Route itself - Eliminates the need for conditional logic to figure out which rule to apply - Easier extend to new claim route rule
Disadvantage	- The resolver requires complex conditional logic to fetch the right strategy, less extensible - Introduce anemic domain model, the Route has no behaviour	- Routes become slightly more complex as they now manage injected behaviour dependencies

Decision: Alternative B is chosen. Alternative A examines a route's data to decide which validation strategy to apply. However, this led to complex conditional logic inside the resolver and separated the validation behaviour from the Route data. Alternative B solves this by injecting the specific RouteClaimStrategy directly into the Route upon creation. This associates behaviour

directly with the object, cleanly eliminating conditional statements and making it trivial to add new route types in the future.

Application 2: Scoring Logic (Route & End-Game)

Alternative	**A:** Hardcoded scoring methods in game/actions	**B:** Strategy pattern (RouteScoringStrategy & EndGameScoringStrategy)
Advantage	- Simpler initial implementation - Fewer classes created	- Easily inject new scoring rules for different maps/expansions without touching existing code - Adheres to Open/Closed Principle - Allow composing multiple end-game rules dynamically
Disadvantage	- Rigid architecture, adding a new map with different point values or rules forces modification of core game logic - Huge switch statements required to handle different rulesets	- Increases the number of interfaces and small classes in the domain

Decision: The future map expansions may introduce unique scoring mechanics. Alternative A will require constant modification of core classes, violating OCP. By implementing scoring strategies, the rules are encapsulated. This allows the EndGameService and action commands to dynamically calculate points by simply executing whatever strategy is injected.

4.5.5 Mediator Pattern

Application 1: Turn Flow

Alternative	**A:** State Pattern (Strict State Transitions)	**B:** Mediator Pattern (Cost-based Coordination)
Advantage	- Clearly models different turn phases - Encapsulates state transitions and flow	- Centralises coordination between commands, player, and action flow - Handles action cost, pending decisions, and turn completion in one place - Avoids a massive class explosion of micro-states
Disadvantage	- Introduces class explosion/overengineering, as a huge number of specific state classes to avoid enums is needed	- Relies on action cost enums means adding new, uniquely behaving actions might require modifying the Mediator - Commands are coupled to the

	- Lead to scattered logic across multiple states if enums are still used to reduce classes, creating high coupling	Mediator's cost/status system

Decision: Alternative B is chosen and preferred. Managing turn flow requires a pragmatic tradeoff. Alternative A requires each ActionCommand to have the knowledge of the next state to have after executing, and each TurnState requires knowledge of which Command is valid to be executed, which highly coupled ActionCommand and TurnState together and breaks Open-Closed Principle, as new ActionCommand added requires modification of each TurnState.

Alternative B is the practical choice. While using an ActionCost system and enums to represent the cost of an Action slightly violate the Open/Closed Principle, it preserves severe over-engineering. The Mediator neatly centralizes coordination, action costs, and turn completion. Since the game's complex rules make conditional checks unavoidable, it is architecturally safer to act as a single source of truth for validation and far easier to maintain them in a single Mediator rather than fragmenting them across messy State hierarchy.

4.5.6 Builder Pattern

Application 1: Game Initialization

Alternative	**A:** Factory Pattern (GameFactory/Component Factories)	**B:** Template + Builder Pattern (GameBuilder & GameSetupDirector)
Advantage	- Less initial boilerplate if the game setup were extremely simple	- Handles complexity and multi-step structurally, GameSetupDirector dictates the sequence assembly, while the concrete builders handles the details - Adding a new map just requires creating a new concrete builder class without modifying existing setup logic.
Disadvantage	- Adding new maps force modifying the core factory class (adding methods for different maps), violating OCP - A single GameFactory would be bloated with large conditional (mapType) logic and helper methods to route creation logic, while splitting it creates an unmanageable mess of microfactories. - Difficult to scale when initialisation need more steps and configuration	- Still requires a localized switch statement to select correct concrete builder based on user input

Decision: Alternative B is chosen. Constructing a Game requires a complex, multi-step initialization sequence. Alternative A is meant for single step creation; using it here would result in poor extensibility, as adding new maps would require modifying existing factory logic, directly violating the Open/Closed principle.

Alternative B allows GameSetupDirector to encapsulate the step-by-step order in a form of template, as setup order for different maps are the same. The Builder interface also makes the system highly extensible, introducing a new map just means writing a new concrete builder without touching the core setup logic. While choosing the concrete builder inherently requires an initial switch statement based on the MapType, isolating this minor Open/Closed principle violation purely at the application's creation boundary is a high pragmatic tradeoff.

4.6 Architectural Trade-offs: MVC Boundaries & Data Transfer

Alternative	**A:** Direct Model Access	**B:** Strict DTO Decoupling
Advantage	- Avoids excessive DTO classes explosion - Simpler and much faster to implement for the assignment scope	- Pure decoupling, the UI has zero dependency on the shape of the domain model
Disadvantage	- Introduces coupling between the View and the internal model structure	- Overengineering for a small local desktop application - Requires creating and mapping read-only or DTO classes for every single entity, adding massive classes explosion

Decision: Alternative A is chosen. While strict decoupling (Alternative B) reduces UI dependency on the domain model, introducing DTOs for every entity in a desktop application of this assignment scope leads to excessive boilerplate. Thus, a minor degree of Model-View coupling is justified with pragmatic simplicity. To preserve the core separation of responsibilities, Views are strictly prohibited from mutating state. These views only read data for rendering or minor UI validation, and actual state modifications are routed through the Controller.

Exception: Selective use of DTOs for Aggregated Data (Post-UML Implementation)

While core models are passed directly to the View, some DTOs are selectively implemented for aggregated data to prevent the View from handling complex logic. Specifically, we created TicketDecisionDTO (for handling temporary destination card draws) and GameResultDTO (for packaging the winner and leaderboard stats). These specific DTO classes were introduced during the final code implementation phase to optimize data transfer, thus they are not reflected in the submitted final UML diagram.

5 Supplementary Documentation

5.1 Test Performed

5.1.1 Integration Test

Integration tests are performed by playing through the game to ensure integration between frontend and backend are producing the result as expected.

Test performed	Outcome	Is Expected
Game setup	Successful deal a starting hand of 5 TrainCard to each Player	True
	Successful deal 3 DestinationTicket to Player, and allowed Player to keep at least 2 DestinationCard	True
	Successful assign the correct number of score (0) and trainPieces (16) to the Player after setup completed	True
Claiming Route	Successful claimed the Route and double-Route	True
	Fail due to insufficient cards	True
	Fail due to mismatch card colour	True
	Fail due to insufficient train pieces	True
	Fail claiming double-route when another parallel route is claimed	True
Drawing train card	Successful claim Locomotive card from face-up as the first action and turn ended	True
	Successful claim non-locomotive card from both deck and face-up	True
	Fail claiming Locomotive from face-up at second draw	True
	Successful draw Locomotive from deck with allowing Player to have a second draw	True
	Successful draw Locomotive from deck when Player performing second draw	True
	Fail performing other Action such as drawing DestinationCard and ClaimRoute before completing second draw	True
Draw Destination Card	Successful draw DestinationCard from deck and prompt user to select at least one card	True

	Fail due to selecting 0 DestinationCard drew from deck	True
End Game	Successful trigger end game when Player remaining train pieces are less than 2 when the turn end	True
	Successful ended the game when each Player takes a final turn after triggering end game	True
	Successful compute the score and show the leaderboard with scores of each Player	True

5.1.2 Unit Test

Unit tests are performed for each of the strategy, service, and model classes.
Path for test file: *project/test*

5.2 Known Defects

Description	Expected results	Actual results
The game enters a stuck state when a player draws a standard train card, and the remaining draw pile has only face up locomotives cards, no valid second draw is possible.	Ending the player's turn automatically	Game become stuck and no further actions is available

5.3 Implementation Limitation

The system contains frontend rendering compatibility issues. Frontend visualisation displays may differ between different machines due to the size and dimension of each machine's screen. Hence, for a better visual experience, the MacBook Air 13.6-inch model is recommended.

5.4 Executable Deliverable

Terminal commands to rebuild and rerun the executable (jar) is included in the README.md file in the git repository.
It is recommended to run on a MacBook Air 13.6-inch Linux environment for a better display experience, other environments are also acceptable, but may have slightly poor UI display experience.

6 AI declaration

We used generative AI (ChatGPT) to create artwork assets for the project, including card PNG images for UI visualisation purposes. The images were further cropped to fit the desired visualisation.

We also used generative AI (ChatGPT) to assist in writing frontend UI code for advanced visual elements. The items were then reviewed, adjusted and integrated to fit project use.

Additionally, generative AI (ChatGPT) was used to assist with grammar correction, sentence rephrasing and improving the clarity of rationale explanations. All generated text was reviewed and further edited by team members.

The AI-generated outputs were reviewed and selectively used. All final design choices were made by the team.